

빅데이터개론

주가 데이터 파이프라인 제작

(수집 → 저장 → 가공 → 시각화 → 확인)

bigd_01_setup : 전체 그림과 Git/GitHub

우리가 만드는 것은 데이터 파이프라인입니다

주가 데이터를 사용합니다

수집 → 저장 → 가공 → 시각화 → 확인

- 수집 : 주가 데이터를 어디선가 가져온다
- 저장 : 가져온 것을 사라지지 않게 쌓아 둔다
- 가공 : 쌓인 것을 다루기 좋은 모양으로 정리한다
- 시각화 : 정리된 것을 한눈에 보이는 그림으로 만든다
- 확인 : 만들어진 것을 내가 원할 때 꺼내 본다

회사에서 하는 데이터 일도 이 다섯 칸을 그대로 지나갑니다.

다루는 양과 도구가 클 뿐 구조는 같습니다. 우리는 그 축소판을 한 바퀴 돕니다.

단계마다 무엇을 쓰는가

이 프로젝트에서 사용하는 기술입니다

단계	쓰는 기술	배우는 장
수집	네이버 금융 API	3장
저장	Turso DB (클라우드 SQLite)	8장
가공	파이썬 문법 · pandas	4 · 5 · 6장
시각화	matplotlib	7장
확인	텔레그램 봇 · Streamlit 대시보드	2 · 12 · 13장
자동 실행	GitHub Actions	10장
코드 보관	Git · GitHub	1장 (그 뒤 매장)

13장 동안 하나씩 붙습니다

한 번에 다 만들지 않습니다

1장 환경 세팅 · 내 저장소 만들기

2장 봇이 내 폰에 말을 건다

3장 현재가가 내게 온다

4·5장 코드 읽기 — 파이썬 문법 · pandas/matplotlib

6장 관심종목 여러 개가 온다

7장 그래프가 온다

8장 종가가 DB에 쌓인다

9장 그래프 왼쪽이 채워진다 · 공용 모듈 분리

10장 가만히 있어도 온다 (자동 실행)

11장 실측 — 가끔 안 온다

12장 웹에서도 본다

13장 누른 것이 감지되어 돌아온다 · 전체 리뷰

매 장마다 ' 지난 장까지 만든 것 + 이번 장 것'을 통째로 받습니다. 처음부터 다시 짜지 않습니다.

받은 코드에 '01-WHATS-NEW.md'가 들어 있습니다. 이번 장에서 뭐가 늘었는지 한 장으로 적혀 있습니다.

왜 완성된 코드를 먼저 주나요?

백지에서 짜지 않습니다

- 이 수업은 새로운 프로그래밍 언어를 배우는 시간이 아닙니다.
- **목표는 데이터 파이프라인이 완성되기까지 점진적으로 발전해 가는 과정을 체험하는 것입니다.**
- 정답이 손에 있으면 '막혀서 못 따라가는' 일이 사라집니다. 누구나 전 과정을 경험할 수 있습니다.
- **대신 코드를 '읽고 이해했는가'를 매 장마다 퀴즈로 확인합니다**
- 퀴즈는 코드를 읽어야만 풀립니다 — 출력 결과를 예측하는 문제들입니다

그래서 영상을 안 보고 퀴즈를 풀면 티가 납니다.

반대로 영상을 보고 코드를 한 번 읽으면 대부분 풀립니다.

13장 뒤에 여러분이 갖게 될 것

지금부터 이것을 향해 갑니다

- 매일 정해진 시각에 텔레그램으로 관심종목 현재가가 온다
- 가격·등락률과 함께 최근 추이 그래프가 사진으로 온다
- 보고 싶을 때는 웹 대시보드를 열어 그 즉시 확인할 수 있다
- **내 컴퓨터를 꺼 두어도 알아서 돌아간다**

핵심은 '파이썬 문법'이 아니라 '돌아가는 것을 하나 만들어 본다'입니다.

최종 결과 미리보기

텔레그램과 대시보드로 오는 메시지

1. 텔레그램

Push (데이터 보내기 / 저장·처리)

- 생성된 결과 데이터를 저장소에 밀어 넣는 (Push) 작업
- 수집한 곳에서 중앙 서버·DB·외부 시스템으로 전송하는 흐름

2. 웹 대시보드

Pull (데이터 가져오기 / 조회·시각화)

- 저장되어 있는 데이터를 불러와 사용자에게 보여주는 흐름
- 내가 열어 볼 때마다 그 시점의 최신 상태가 보임



이제 Git과 GitHub 이야기입니다

이번 장에서 실제로 할 일

왜 GitHub이 필요한가요?

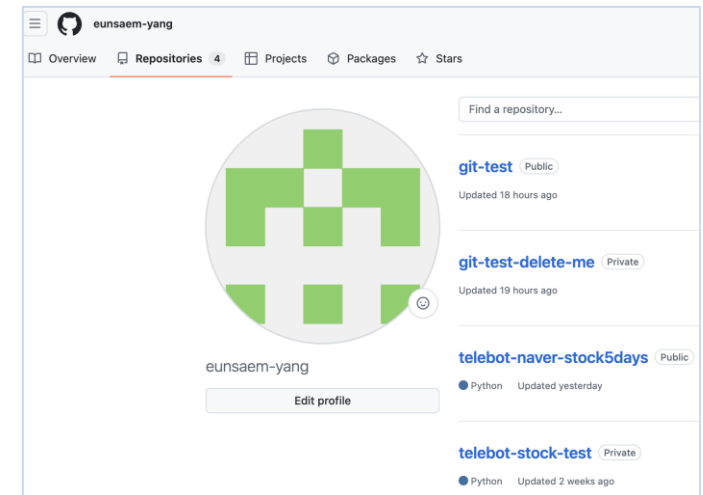
10장에 GitHub가 내 코드를 대신 실행하려면, 그 코드가 거기 있어야 합니다

- 매 장마다 받은 코드를 어딘가에 두어야 합니다
- 그때 코드를 실행해 주는 곳이 GitHub입니다
- **10장부터는 '내 컴퓨터가 꺼져 있어도' 코드가 돌아야 합니다**
- **그래서 이번 장에서 저장소를 만들고, 2장부터 매 장마다 GitHub에 올립니다**

GitHub 저장소(Repository)

프로젝트 하나 = 저장소 하나

- 이 수업에서 만드는 프로젝트 전체를 담는 폴더라고 생각하면 됩니다
- 공개(public) / 비공개(private) 중 하나를 고릅니다 — 둘 다 됩니다
- 나중에 바꿀 수 있습니다



고를 때 알아둘 것

- 이 수업에서는 public을 권합니다 — private은 12장에 권한 승인 단계가 하나 더 있습니다
- public이라서 .env(비밀번호가 든 파일)를 올리지 않는 것이 더 중요해집니다

커밋(Commit)

'여기까지 됐다'고 도장을 찍는 것

- 파일을 바꾼 뒤 '이 상태를 기록해 둔다'고 표시하는 행위입니다
- 커밋마다 '무엇을 왜 바꿨는지' 메시지를 남깁니다
- 기록이 쌓이므로 언제 무엇이 바뀌었는지 나중에 볼 수 있습니다

이번 장에서는 VS Code의 Commit 버튼 하나로 합니다.

2장부터는 매 장마다 Commit → Sync 두 버튼만 반복합니다. 터미널 명령은 13장 내내 쓰지 않습니다.

가장 중요한 개념

원격(GitHub)과 로컬(내 컴퓨터)은 별개의 사본입니다

내 컴퓨터 (로컬)

- 압축을 푼 폴더
- 여기서 코드를 실행합니다

GitHub (원격)

- 웹에 올려둔 사본
- 10장부터 여기서 코드가 돌아갑니다

한쪽을 바꿔도 다른 쪽은 그대로입니다.

웹에서 파일을 고쳐도 내 컴퓨터 파일은 안 바뀝니다. 그 반대도 마찬가지입니다.

개발 환경 갖추기

이번 장에서 한 번만 설치합니다

압축파일을 푼다

내 컴퓨터에 설치

• 받은 압축파일을 풉니다.

- 학기말까지 계속 사용할 폴더이므로 위치를 정해두고 압축을 풉니다
- **폴더의 경로에 한글이 섞이지 않게 합니다**
- 폴더 안에

FAQ.md : 「막혔을 때」 1절은 장별 세팅, 2절은 증상으로 원인 찾기용 설명 파일입니다

watchlist.csv : 관심 종목 코드와 종목명이 들어있습니다

: vscode 또는 메모장(마지막 종목 줄 뒤에서 Enter 한 번 치기)으로 수정 가능(인코딩 UTF-8)

.gitignore : 깃에 올라가면 안되는 파일(특히 .env 파일)들의 목록이 들어있습니다. **중요!!!**

: **GitHub 한 번 커밋되면 되돌릴 수 없습니다.**

숨김 파일의 기준은 맥과 윈도우가 다릅니다

macOS Finder — 이름이 점으로 시작하면 숨깁니다 (Cmd+Shift+. 으로 볼 수 있습니다)

윈도우 탐색기 — 점은 상관없습니다. .gitignore 도 그냥 보입니다 (안보이면 탐색기에서 숨김파일 표시를 하면 됩니다)

폴더는 두 개를 씁니다

학기 내내 쓰는 폴더 하나 + 매 장마다 zip을 푸는 폴더 하나

메인 프로젝트 폴더

- 학기 내내 쓰는 폴더 하나
- 여기에만 GitHub을 연결합니다
- 매 장마다 이 폴더의 내용이 갱신됩니다

매 장별 폴더 (매 장마다 새로 생김)

- 그 장에서 받은 zip을 푸는 곳
- 여기 있는 것을 전부 복사해서 메인 폴더에 덮어씁니다

⚠ 메인 폴더 이름은 반드시 영문·숫자로 만드세요.

이름이 한글이면 저장소 이름이 하이픈으로 깨집니다 — 빅데이터프로젝트 → -----

잠시 뒤 27장에서 실제 화면으로 봅니다. 예: bigdata-project · stock-2026

Python 설치

체크 하나를 놓치면 터미널이 파이썬을 못 찾습니다

- **먼저 이미 깔려 있는지 터미널에서 확인합니다**

`python --version` (macOS: `python3 --version`)

- 3.11보다 낮거나 없으면 python.org/downloads 에서 받습니다

- **윈도우는 설치 화면 맨 아래 Add python.exe to PATH 체크가 필수입니다**

체크하지 않으면 터미널이 python 을 못찾는다 (FAQ.md 2절 11번)

- macOS는 python3 로 확인합니다(python 만 치면 옛 버전이 잡힐 수 있습니다)

확인 명령이 3.11 이상을 표시하면 이 단계는 끝입니다.

이미 설치되어 있다면 새로 설치 할 필요는 없습니다.

VS Code 설치

코드를 쓰고 읽는 편집기입니다

- 받는 곳 — code.visualstudio.com

설치 화면은 기본값 그대로 넘어가면 됩니다

- vscode Extension 설치

확장 : Python

게시자 : Microsoft

비고 : Pylance(자동완성·문법 검사)와 Python Debugger가 같이 달려 옵니다

- Code Runner는 설치하지 않는다

.env를 찾지 못하는 문제가 생깁니다

- vscode – **파일메뉴 – 폴더열기(Open folder)**로 메인 프로젝트 폴더를 엽니다.

vscode 폴더 안에 FAQ.md, watchlist.csv 등이 보입니다

라이브러리 7개를 한 번에

13장 동안 이 설치하는 이번 한 번뿐입니다

- 터미널에서 이 한 줄을 그대로 칩니다

`pip install requests python-dotenv pandas matplotlib holidays streamlit libsql-client`

- macOS는 pip 대신 pip3 를 사용합니다

- holidays 는 주말·공휴일을 걸러 그날 증시가 열리는지 판단할 때 사용합니다 (10장)

- 한 번 설치하면 13장 내내 다시 설치할 일이 없습니다

- 2장 누적본 부터 해당 장에 필요한 라이브러리 목록은 requirements.txt에 들어있습니다

`pip install -r requirements.txt` 를 사용할 수 있습니다.

설치나 계정 만들기가 있는 장은 네 번뿐입니다.

1장 — Git · GitHub · 2장 — 텔레그램 봇 · 8장 — Turso DB · 12장 — Streamlit

GitHub 계정 만들기

저장소는 잠시 뒤 VS Code에서 만듭니다

- **GitHub 계정 — 이 수업 전체의 통합 로그인 역할이라 필요합니다**

- github.com/signup 에서 계정을 생성합니다
- 이후에 나오는 외부 서비스 로그인이 전부 GitHub 계정을 사용합니다
: 8장 Turso DB, 12장 Streamlit Community Cloud
- 가입 직후 오는 인증 메일을 꼭 확인해 두세요

- **저장소(Repository)는 GitHub 웹에서 만들지 않습니다**

- VS Code에서 버튼 하나로 만듭니다 — 저장소 생성과 첫 업로드가 한 번에 끝납니다
- 지금은 계정만 준비돼 있으면 됩니다

웹에서 저장소를 미리 만들지 마세요.

같은 이름이 이미 있으면 VS Code 쪽이 조용히 실패합니다(28장).

Git 프로그램

VS Code의 소스 제어는 이 프로그램을 불러서 씁니다

- **VS Code 버튼 뒤에서 실제로 일하는 것이 Git입니다**
- 윈도우 : git-scm.com 에서 Git for Windows 를 받아 기본값 그대로 설치합니다
- macOS : 터미널에 git --version 을 쳐 봅니다

설치 안내 창이 뜨면 Xcode Command Line Tools 를 설치하면 됩니다

- **설치한 뒤에는 VS Code를 켜다 켜야 소스 제어가 Git을 찾습니다**

설치와 설정이 끝나면 조작은 전부 VS Code 버튼으로 합니다.

터미널을 여는 것은 파이썬 확인 · git 확인 · 이름·이메일 설정 · 라이브러리 설치, 이렇게 넷뿐입니다.

Git에게 내가 누구인지 알려줍니다

확인 → 없으면 설정 → 다시 확인. 한 번만 하면 이 컴퓨터의 모든 프로젝트에 적용됩니다

- 커밋에는 누가 했는지가 함께 기록됩니다 — 그래서 이름과 이메일이 필요합니다
- **먼저 첫 줄만 쳐 봅니다** — 아무것도 안 나오면 아직 설정되지 않은 것입니다
git config --global user.name //값이 나오면 이미 돼 있는 것이니 다음으로 넘어가도 됩니다
git config --global user.name "**내_깃허브_아이디**" //위 명령으로 값이 나오지 않으면 설정합니다
- **⚠ 내_깃허브_아이디 · 내_이메일은 각자 자기 것을 넣습니다.** 화면에 보이는 대로 복사해 붙이면 안 됩니다
- **이메일을 설정합니다 ⚠ 이메일은 GitHub 계정 이메일과 같게 씁니다.** 다르면 커밋이 내 것으로 잡히지 않습니다
git config --global user.email "**내_이메일**@example.com"

이걸 안 하면 다음 장에서 Author identity unknown 으로 막힙니다.

“Please tell me who you are” — 원인이 화면에 드러나지 않으니 여기서 미리 해 둡니다.

내 저장소 만들기 — 버튼 하나로 끝냅니다

.gitignore를 만들고 → Publish to GitHub 를 누릅니다

.gitignore를 직접 만듭니다

복사하지 않습니다 — 이 파일이 무엇을 막는지 알고 넘어가야 합니다

① VS Code 왼쪽 탐색기에서
새 파일 아이콘을 누릅니다

② 이름을 .gitignore 로 칩니다
(점으로 시작합니다)

③ 오른쪽 내용을 그대로 칩니다

④ **Ctrl+S (macOS ⌘S) 로 저장**

· 맥에서는 숨김 파일이라 안 보이고,
윈도우에서는 그냥 보입니다

비밀번호 — 절대 커밋하지 말 것

```
.env  
.env.*  
!.env.example  
*token*.txt  
*secret*
```

```
__pycache__/  
*.pyc  
*.docx  
*.pptx  
*.pdf  
*.jpg  
*.jpeg  
*.png
```

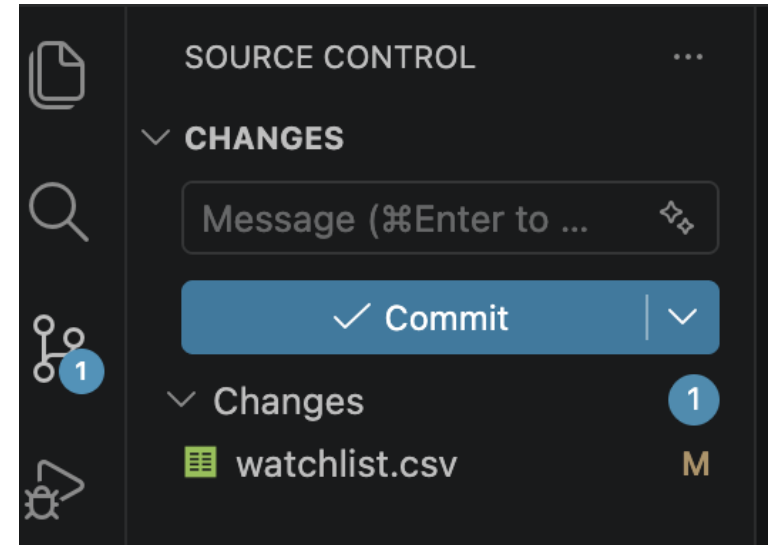
받은 zip 안에도 같은 파일이 있지만, 복사하지 말고 여기서 직접 만듭니다.

틀리면 조용히 실패하는 파일이라, 무엇을 막고 있는지 한 번 보고 넘어가야 합니다.

왜 .gitignore가 먼저인가요

토큰이 생기기 전에 방어막을 세웁니다

- 이 파일은 'GitHub에 올리지 않을 것'의 목록입니다
- 2장에 만들 .env 에는 텔레그램 봇 토큰이 들어갑니다
- 8장에는 Turso DB 토큰도 여기에 들어갑니다
- **한 번 커밋되면 GitHub 이력에 남아 되돌릴 수 없습니다**
- **지금은 .env 가 아직 없습니다 — 그래서 지금 만드는 것입니다**



커밋 대상 목록(Changes)에
.env 가 아예 뜨지 않습니다

2장에 .env를 만들면 이 목록에 안 뜹니다. 그 '왜 안 뜨지'가 이 파일이 일하고 있다는 증거입니다.

① VS Code로 메인 폴더를 엽니다

폴더를 처음 열면 신뢰할지 묻습니다

- 파일메뉴 → **폴더 열기(Open Folder)**로 **메인 프로젝트 폴더**를 엽니다
- '이 폴더의 파일 작성자를 신뢰합니까'를 묻는 화면이 뜹니다
- 신뢰함(Yes, I trust the authors)을 선택해야 소스 제어가 살아납니다
- **신뢰하지 않으면 Manage Workspace Trust 버튼만 보이고 Publish to GitHub는 아예 없습니다**

버튼이 없는 이유가 화면에 나오지 않습니다 — 버튼이 안 보이면 여기부터 확인하세요.

나중에 바꾸려면 파일 → 작업 영역 신뢰 관리(Manage Workspace Trust)에서 할 수 있습니다.

② 소스 제어 패널을 엽니다

버튼이 두 개 보입니다 — 아래쪽을 누릅니다

- 왼쪽 세로 아이콘 줄에서 소스 제어를 엽니다

단축키 Ctrl+Shift+G (macOS ^⇧G)

- ⚠️❌ 위의 **Initialize Repository** 를 누르지 마세요

실수로 위 버튼을 눌렀다면 — .git 폴더를 지우면 처음 상태로 돌아갑니다

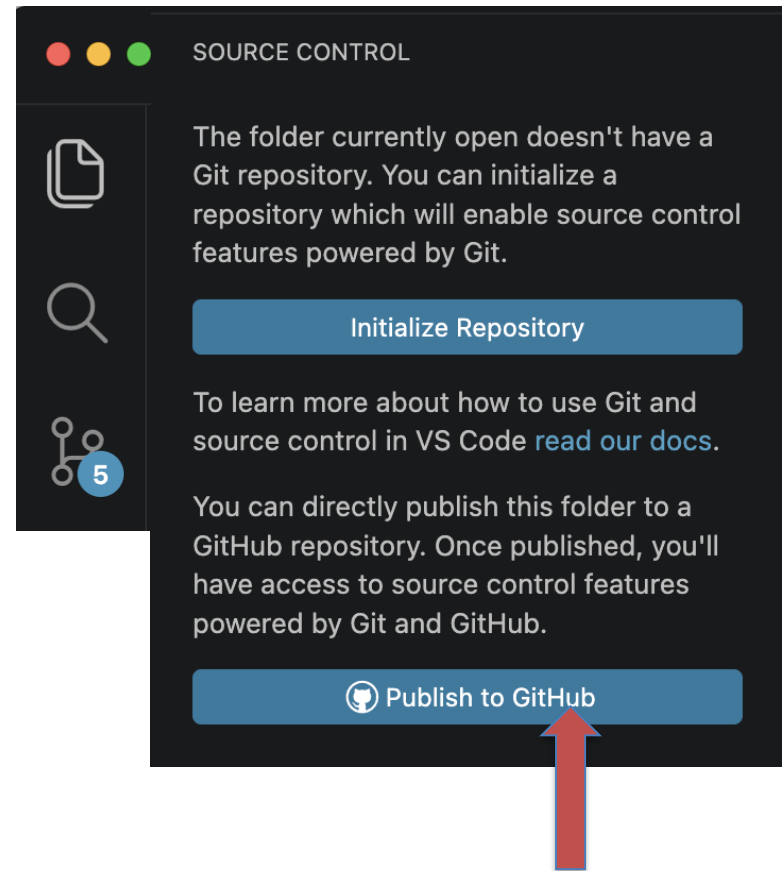
① VS Code를 닫고 ②숨김 파일을 켜고 .git 폴더만 지웁니다 ③ 폴더를 다시 엽니다(숨김 파일 켜기 — Finder ⌘⇧. / 탐색기 보기 → 숨긴 항목)

- 아래 **Publish to GitHub** 를 누릅니다

⚠️❌ 이때 로그인 창이 뜹니다 — 윈도우는 두 번 뜹니다 (30장)

위 버튼을 먼저 누르면 브랜치 이름 맞추기와 첫 커밋이 건너뛰어집니다.

그러면 윈도우에서는 브랜치가 master 로 만들어져 10장 자동 실행이 돌지 않습니다.

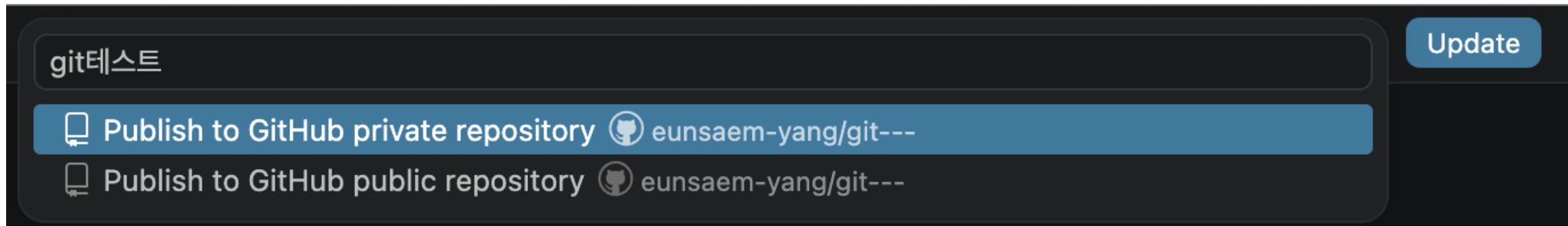


③ 저장소 이름은 영문·숫자로

폴더 이름이 그대로 채워져 나옵니다

⚠️ 한글은 전부 하이픈(-)으로 바뀝니다

입력칸에는 한글이 그대로 보이고, 바뀐 이름은 오른쪽 작은 회색 글씨에만 나옵니다.
그래서 모르고 그냥 넘어갈 수 있습니다.



↑ 입력칸은 git테스트 인데, 오른쪽 회색 글씨는 eunsaem-yang/git--- 입니다.

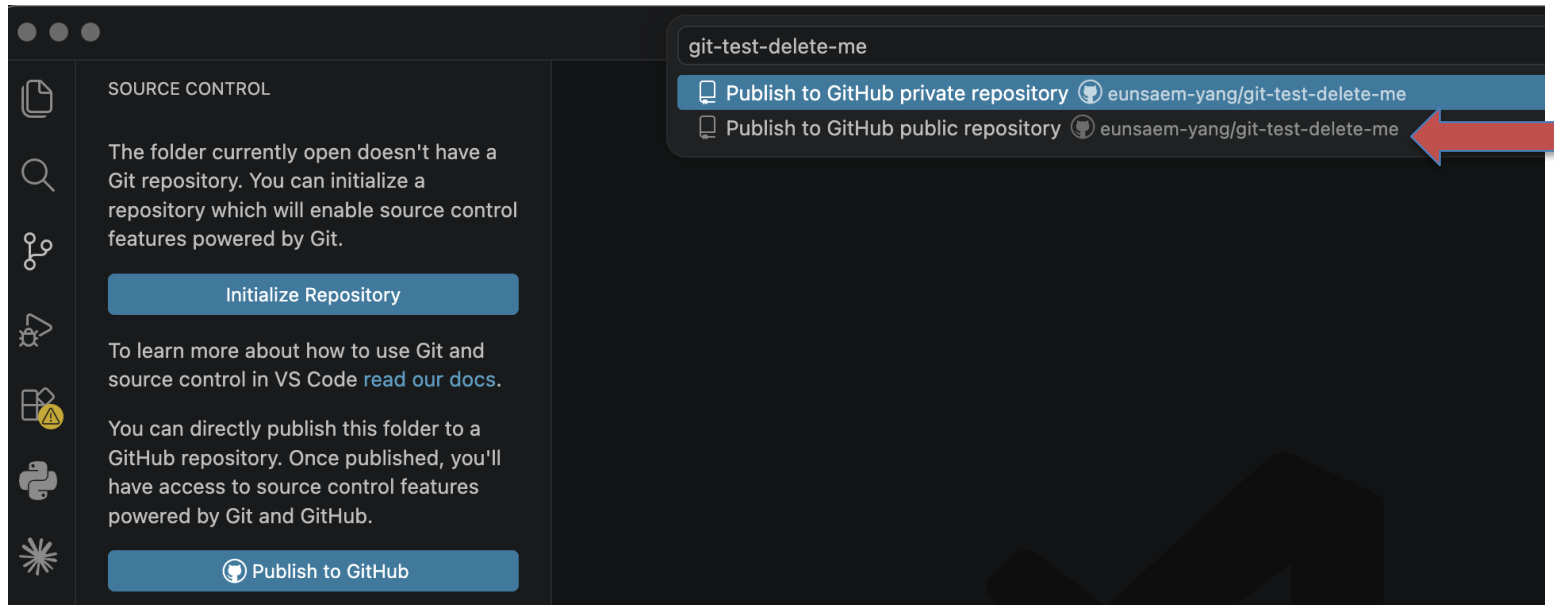
15장에서 메인 폴더 이름을 영문·숫자로 만들어 두었다면 이 문제는 생기지 않습니다.

빅데이터프로젝트 라는 폴더로 하면 저장소 이름이 ----- 이 됩니다.

④ 아래 public 을 클릭합니다

그냥 엔터를 치면 private 가 됩니다

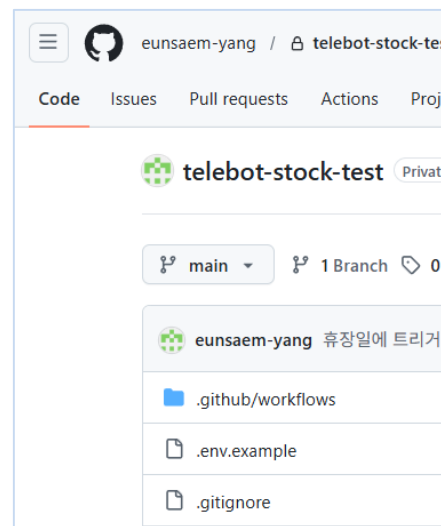
- 이름을 친 뒤 아래에 항목이 두 개 나옵니다
- 위 **private repository** 가 기본 선택입니다 (파란 줄)
- ⚠️❌ 아래 **public repository** 를 마우스로 클릭하세요



⑤ 승인하면 저장소가 만들어집니다

생성 · 첫 커밋 · 업로드가 한 번에 끝납니다

- GitHub 로그인 창이 나오면 브라우저 안내에 따라 승인합니다
- ⚠ 윈도우는 여기서 로그인 창이 한 번 더 뜹니다 — 다음 장
- 돌아오면 저장소 생성 · 첫 커밋 · 업로드가 이미 끝나 있습니다
- ⚠ 아무 일 없이 처음 화면으로 돌아왔다면 같은 이름의 저장소가 이미 있는 것입니다
- 에러가 눈에 잘 띄지 않으니 주의하세요



윈도우는 로그인 창이 두 번 뜹니다

맥은 한 번입니다 — 두 번째 창은 고장이 아닙니다

	인증을 누가 하나	로그인 창
macOS	VS Code가 둘 다 처리합니다	한 번
Windows	만들기는 VS Code, 올리기는 Git 로그인	두 번

윈도우에서 실제로 보이는 순서

Publish to GitHub → ① Connect to GitHub (사용자명·비밀번호) → Continue

→ 저장소 이름 입력 → public 클릭

→ ② **Sign in with your browser** → 브라우저 안내에 따라 승인

「아까 로그인했는데 왜 또?» — 하는 일이 다른 두 번입니다. 둘 다 승인하세요.

한 번 해 두면 13장 동안 다시 묻지 않습니다.

2장부터 — 매 장마다 두 버튼

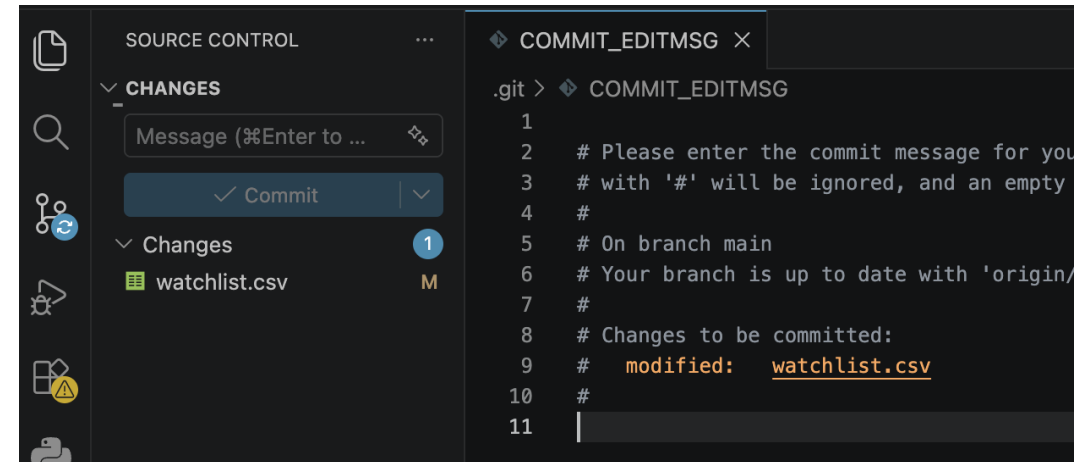
덮어쓰기 → Commit → Sync. 이게 전부입니다

매 장이 끝나면 하는 일은 이 다섯 단계입니다

2장부터 13장까지 매 장이 같습니다

- ① 그 장에 해당하는 zip을 풀어 전부 복사 → 메인 폴더에 덮어씁니다
- ② 소스 제어 패널을 열면 바뀐 파일이 목록에 뜹니다
- ③ 맨 위 한 줄 입력칸에 커밋 메시지를 씁니다. 메시지 칸에 입력이 안될 때는 그 옆 **commit_editmsg**의 1번 줄에 커밋 메시지를 씁니다.

- ④ 그 아래 파란 Commit 버튼을 누릅니다
- ⑤ 버튼이 Sync Changes 로 바뀌면 한 번 더 누릅니다



매 장마다 이 다섯 단계가 전부입니다. 터미널 명령은 하나도 없습니다.

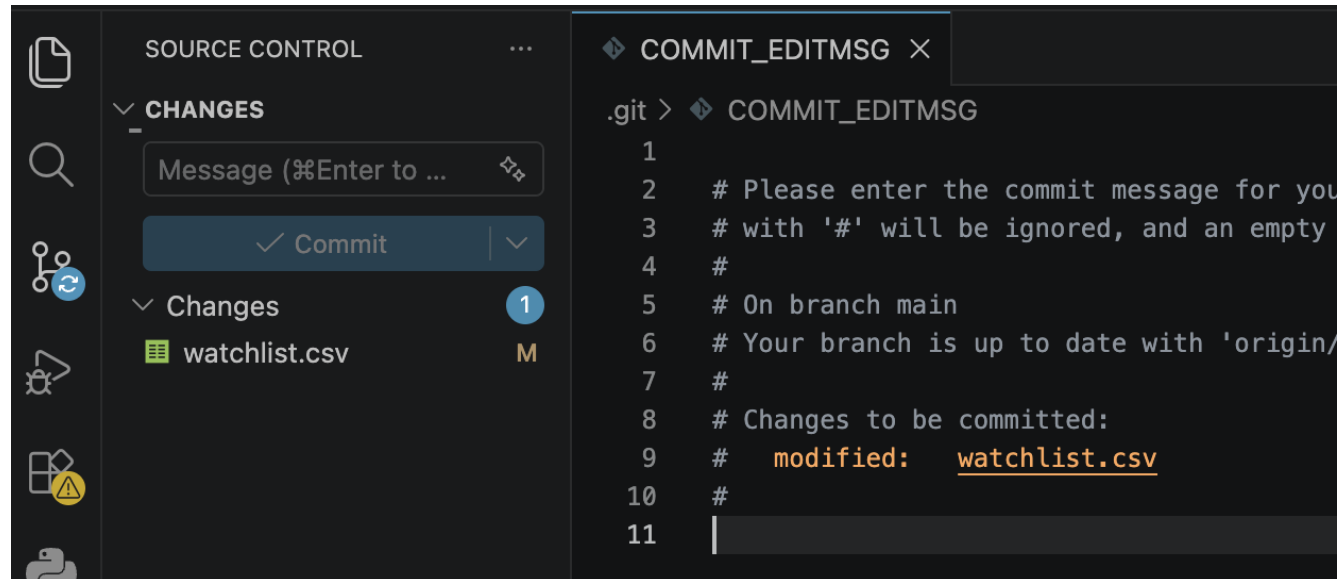
커밋 메시지는 '1장 코드' 처럼 짧게 이해하기 쉽게 적으면 됩니다.

↑ 메시지 칸은 여기,
Commit 버튼은 바로 그 아래

메시지를 먼저 쓰고 Commit 을 누릅니다

순서를 바꾸면 낫선 편집기가 열립니다

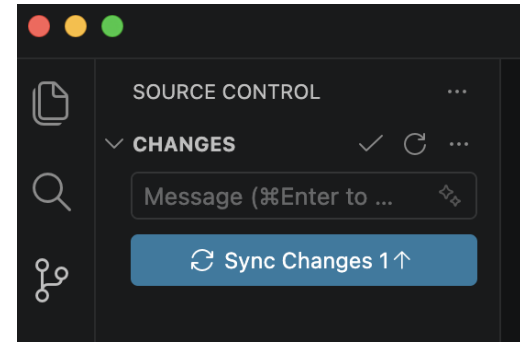
- 메시지를 비운 채 누르면 COMMIT_EDITMSG 가 열립니다
- 그동안 소스 제어 패널은 잠깁니다 (Git이 기다리는 중)
- **탈출법 — 1번 줄에 메시지 입력 → ⌘S 저장 → ⌘W 로 탭 닫기**
- 왼쪽 Changes 를 보세요 — .env 가 없습니다



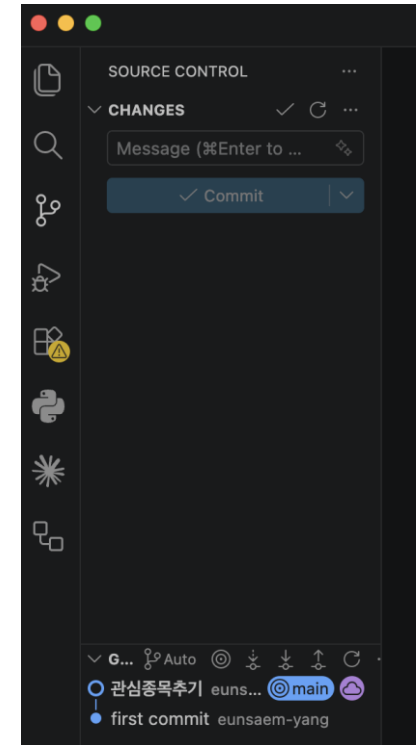
Sync Changes 를 눌러야 GitHub으로 갑니다

Commit 만 하면 아직 내 컴퓨터 안입니다

- 커밋은 '내 컴퓨터에 도장을 찍은 것'까지입니다
- 커밋하면 버튼이 Sync Changes 1 ↑ 로 바뀝니다
- 누르면 확인 대화상자가 한 번 뜹니다 — OK
- 끝나면 Changes 가 비고, 아래 그래프에 커밋이 쌓입니다
- 12장의 '원격과 로컬'이 맞춰지는 순간입니다



↑ 커밋한 뒤 나타나는 버튼



↑ 끝난 뒤 — Changes가 비었다

텔레그램

- 텔레그램 앱 설치와 계정 만들기는 2장 영상에서 설명한 후, 2장에 사용합니다.

이번 장에서 할 일

- 받은 압축파일 풀기 · 메인 프로젝트 폴더 만들기 (이름은 영문·숫자)
 - Python 3.11 이상이 깔려 있는지 확인
 - VS Code 설치 + 확장 Python(게시자 Microsoft) 설치
 - Git 프로그램 설치 (윈도우 Git for Windows / macOS 안내대로)
 - GitHub 계정 만들기
 - **메인 폴더에 .gitignore 를 직접 만들기**
 - **VS Code 소스 제어에서 Publish to GitHub 로 저장소 만들기**
 - 필요한 라이브러리 7개를 pip install 로 한 번에 설치 (13장 동안 한 번뿐)
 - **대면 실습 귀가조건 : 자신의 것을 다 하신 분은 누구든 한 명 도와주고 가기**
- 제출물: 내 GitHub 저장소에 파일이 올라간 화면 캡처 1장**

막혔을 때

혼자 오래 붙잡지 마세요

- 받은 압축파일 안에 FAQ.md 가 들어 있습니다

- : 1절 — 이번 장에서 무엇을 세팅해야 하는지

- : 2절 — 내가 본 오류 메시지로 원인 찾기

- 오류 메시지를 Ctrl+F 로 그대로 검색해 보세요

- 그래도 안 되면 대면 시간에 물어보세요

1장 정리

- 13장에 걸쳐 '돌아가는 것' 하나를 만듭니다
- 매 장마다 완성된 코드를 받고, 읽고, 이해하고, 핵심 부분만 직접 코딩합니다
- 이번 장은 환경 세팅과 내 저장소 만들기까지입니다
- 2장부터는 매 장이 끝날 때마다 **Commit → Sync** 두 버튼만 반복합니다
- **원격과 로컬은 별개의 사본이라는 것만 기억해 두세요**